

Maestro / MPTK Integration (Virtual Devices + Adapters)

This page documents the optional **Maestro / MPTK (MidiPlayerTK)** integration layer.

It provides two high-level workflows:

1. **MPTK as a virtual MIDI output device (sink)**

Route `MidiManager` events into an MPTK synth (realtime playback).

2. **MPTK as a virtual MIDI input device (source)**

Take MPTK callbacks (from a `MidiFilePlayer` or `MidiStreamPlayer`) and **inject** equivalent MIDI events into `MidiManager` as if they came from a device.

Terminology note: in this plugin, “virtual device” means a software endpoint that participates in `MidiManager`’s device lists and event pipeline without involving platform MIDI backends.

Enable / Disable (compile-time)

The integration is gated behind scripting defines:

- `FEATURE_USE_MPTK`
- `MPTK_PRO` (Maestro Pro APIs: `OnMidiEvent` pipeline, Writer, InnerLoop, ListPlayer, Spatializer, etc.)

When `FEATURE_USE_MPTK` is **not** present (or MPTK is not installed), the integration scripts compile to small stubs so the project still builds. Pro-only APIs are further gated with `#if MPTK_PRO`.

Integration sources live under `Assets/MIDI/Scripts/Integrations/MPTK/` (same layout as Chunity under `Integrations/`).

How to enable

In Unity:

- **Project Settings** → **Player** → **Other Settings** → **Script Compilation** → **Scripting Define Symbols**
- Add: `FEATURE_USE_MPTK`
- When using Maestro Pro, also add: `MPTK_PRO`

Compatible Maestro version

Integration testing baseline: **Maestro / MidiPlayerTK 2.21.x** (Free or Pro). Older 2.15–2.20 builds may work for Free paths; Pro helpers that wrap 2.17+ APIs (voice pause/resume, orientation, chord progression, realtime SoundFont effects) expect 2.21-class Pro.

Runtime helpers (components / MCP)

Helper	Role	Define / MCP
MptkSynthEffectsController	SoundFont filter / reverb / chorus at runtime	MPTK_PRO · midi-mptk-effects-configure
MptkVoiceLifecycle / MptkMidiDevice.PauseVoices	Smooth voice mute / unmute	MPTK_PRO · midi-mptk-voice-lifecycle
MptkDistanceAudioSettings	Distance attenuation + Pro orientation	Free distance · Pro orientation · midi-mptk-distance-audio
MptkChordProgressionPlayer	Play Maestro progression presets	MPTK_PRO · midi-mptk-chord-progression
Global knobs (MptkUtility / MptkBootstrap)	MPTK_RunInBackground , MPTK_AudioListener (optional on EnsureReady)	FEATURE_USE_MPTK · midi-mptk-global-settings
Experimental velocity curve	MPTK_VelocityAttenuation via MptkUtility (use with care)	FEATURE_USE_MPTK · midi-mptk-velocity-attenuation
Visual Scripting / Timeline	Effects, voice, distance, chord, global units; stream control markers	FEATURE_USE_VISUALSCRIPTING / FEATURE_USE_TIMELINE

Diagnostics: `midi://mptk/setup-report` (voice stats / SoundFont / global knobs). Drum presets: `MptkUtility.BuildDrumPresetsText()` , `midi://mptk/soundfont` , or `midi-mptk-bank-program` `action=list-drums` .

Effects and orientation may raise CPU use or lower perceived loudness; keep them off unless needed and adjust Global Volume if required.

What's included

Virtual device registration + input injection

`MidiManager` has helper APIs for software endpoints:

- Register/unregister a virtual device ID (as input, output, or both)
- Inject MIDI events directly into the internal pipeline ("pretend this device sent a Note On", etc.)

Use these when you want to simulate devices, bridge other systems, or unit-test handlers.

MPTK virtual MIDI device (sink)

`MptkMidiDevice` implements MIDI event handler interfaces and forwards events to an internal MPTK `MidiStreamPlayer`.

Use this when you want:

- `MidiManager` → MPTK synth playback
- A "software output device" that appears alongside real devices

MPTK → MidiManager adapter (source)

`MptkMidiManagerInputAdapter` can subscribe to MPTK callback events (commonly exposed by MPTK players) and inject matching MIDI 1.0-style events into `MidiManager`.

Use this when you want:

- MPTK playback / realtime generation to drive your existing `MidiManager` handlers
- MPTK to behave like a virtual *input* device

Convenience utilities

`MptkUtility` wraps common setup:

- Ensuring MPTK globals exist
- Creating MPTK players
- Creating + registering virtual devices
- Creating adapters
- Helper methods to inject MIDI events as virtual input
- Setup validation (`ValidateSetup`)

Pre-synthesis rewrite (`MptkMidiEventPipeline` , **Maestro Pro**)

keep / drop / inject via `OnMidiEvent`. See " `MptkMidiEventPipeline` " in the Bootstrap / diagnostics section below for details.

Writer / external URI (`MptkWriterBridge` , `MptkExternalPlayback` , **Maestro Pro**)

Thin wrappers around `MPTKWriter` / `MidiExternalPlayer`. See " `MptkWriterBridge` / `MptkExternalPlayback` " below for details.

Inner loop (`MptkInnerLoopController` , **Maestro Pro**)

Thin wrapper around `MPTK_InnerLoop` . See “loop region” in the Clock sync section below for details.

Bootstrap / SmfPlayer / diagnostics

Simplifies setup, connects to SMF playback, and provides Panic / Bank Select and debugging aids.

MptkBootstrap — one-click startup

MptkBootstrap automatically does the following:

- Ensures a `MidiPlayerGlobal`
- Registers a virtual device ID (input/output)
- Creates an `MptkMidiDevice` and registers it as a `MidiManager` handler

```
#if FEATURE_USE_MPTK
using jp.kshoji.unity.midi.mptk;
using UnityEngine;

public sealed class MyMptkSetup : MonoBehaviour
{
    void Start()
    {
        var bootstrap = gameObject.AddComponent<MptkBootstrap>();
        bootstrap.EnsureReady();
    }
}
#endif
```

Recommended default ID: `MptkUtility.DefaultOutputDeviceId` (`mptk:internal`)

MptkSmfPlayerOutput — SmfPlayer integration

Automatically sets the `SmfPlayer` `outputDeviceId` to the MPTK virtual device.

```
#if FEATURE_USE_MPTK
using jp.kshoji.unity.midi;
using jp.kshoji.unity.midi.mptk;
using UnityEngine;

[RequireComponent(typeof(SmfPlayer))]
public sealed class MySmfWithMptk : MonoBehaviour
{
    void Awake()
    {
        var output = gameObject.AddComponent<MptkSmfPlayerOutput>();
        output.Configure(); // SmfPlayer.outputDeviceId = "mptk:internal"
    }
}
#endif
```

Placing `MptkBootstrap` on the same GameObject lets `MptkSmfPlayerOutput` reuse it.

MptkDspUmpSequenceOutput — Scriptable Audio UMP integration

When both `FEATURE_USE_MPTK` and `FEATURE_SCRIPTABLE_AUDIO` are enabled, you can route DSP-scheduled UMP through `MidiDspUmpMidi2OutBridge` to the MPTK virtual output.

Sources live under `Assets/MIDI/Scripts/Integrations/MPTK/ScriptableAudio/` (assembly `jp.kshoji.midi.mptk.scriptableaudio`), matching the Chunity / Timeline optional-subfolder layout.

1. Add `MptkDspUmpSequenceOutput` to the same `GameObject` as the UMP Bootstrap / Scheduler (or its wiring target)
2. Optionally place `MptkBootstrap` alongside it (there is an auto-create option when missing)
3. In Play mode, `Configure()` (or `autoConfigureOnAwake`) sets the UMP Out destination to `mptk:internal` etc.

If you do not want double playback with the built-in reference synth, enable the component's `muteBuiltInSynthWhenActive`. See [UMP sequence playback \(DSP sync\)](#) for details.

Panic / All Notes Off / Reset

`MptkMidiDevice.PanicAll()` does the following:

- Stops tracked Note Ons with `MPTK_StopEvent`
- Sends CC 120 (All Sound Off) / CC 123 (All Notes Off) to all channels

When CC 120 / 121 / 123 is received, the equivalent is done per channel. On MIDI Reset (`OnMidiReset`), `PanicAll()` is called.

You can also run the same operation from `MptkBootstrap.Panic()`. Set the destination `deviceId` to `mptk:internal` etc.

Bank Select + Program Change

`MptkMidiDevice` / `MptkMidi2Device` track CC 0 (Bank MSB) / CC 32 (Bank LSB) per channel, and on Program Change send `MPTKController.BankSelectMsb` + `MPTKCommand.PatchChange` to MPTK. Timbre switching stays stable even with non-General-MIDI SoundFonts.

Setup validation (`MptkUtility.ValidateSetup`)

You can check the state of the MPTK integration during Play mode.

```
var report = MptkUtility.ValidateSetup(bootstrap.Sink, bootstrap.DeviceId);
Debug.Log(report.ToSummary());
```

Check item	Severity
<code>MidiPlayerGlobal</code> exists	Error
<code>MidiManager</code> available	Error
Virtual device registered	Error
<code>MidiStreamPlayer</code> / <code>AudioSource</code>	Error / Warning
SoundFont loaded	Error
Applied SF name (when <code>expectedSoundFontName</code> is specified)	Warning

When `autoValidateOnStart` is enabled, `MptkBootstrap` validates automatically at startup.

Debugging: `MptkEventTap`

An optional component that logs MIDI events on the bridge to the Console.

Property	Description
<code>direction</code>	<code>ToMptk</code> (sink direction) / <code>FromMptk</code> (adapter injection direction)
<code>filterDeviceId</code>	Log only a specific deviceId
<code>logNoteEvents</code> / <code>logControlChange</code>	Event-type filters

Add one to the scene and set `isLoggingEnabled = true` to enable logging from `MptkMidiDevice` / `MptkMidiManagerInputAdapter`.

`MptkMidiEventPipeline` — pre-synthesis rewrite (Maestro Pro)

Hooks the `OnMidiEvent` of `MidiFilePlayer` / `MidiExternalPlayer` so you can keep / drop / inject without editing the SMF.

Item	Detail
Defines	<code>FEATURE_USE_MPTK</code> + <code>MPTK_PRO</code> (<code>OnMidiEvent</code> / <code>PlayDirect</code> are Pro)
Component	<code>MptkMidiEventPipeline</code>
Mapping SO	Create > MIDI > MPTK > Event Mapping (<code>MptkMidiEventMapping</code>)
Sample	<code>Assets/MIDI/Samples/MPTK/Scripts/MptkMidiEventPipelineSample.cs</code>

Built-in toggle examples: arpeggio inject, PatchChange drop, SetTempo randomization. On the code side, `Filter` (non-main thread — no Unity APIs). The Mapping’s UnityEvents are queued to the main thread.

```
#if FEATURE_USE_MPTK && MPTK_PRO
using jp.kshoji.unity.midi.mptk;
using MidiPlayerTK;
using UnityEngine;

public sealed class MyPipelineSetup : MonoBehaviour
{
    public MidiFilePlayer filePlayer;

    void Start()
    {
        var pipeline = gameObject.AddComponent<MptkMidiEventPipeline>();
        pipeline.Source = filePlayer;
        pipeline.EnableArpeggio = true;
        pipeline.Filter = e =>
            e.Command == MPTKCommand.NoteOn && e.Channel == 9
                ? MptkMidiEventPipeline.Result.Drop
                : MptkMidiEventPipeline.Result.Keep;
    }
}
#endif
```

When combining with the InputAdapter, route only the **post**rewrite events (`OnEventNotesMidi`) to the virtual input to avoid double notes.

MptkWriterBridge / MptkExternalPlayback — Writer / external URI (Maestro Pro)

Item	Detail
Defines	FEATURE_USE_MPTK + MPTK_PRO (MPTKWriter / MidiExternalPlayer)
Writer	MptkWriterBridge — MidiSequenceAsset / SMF bytes / MidiDB / ImportFromPlayer → Write / in-memory Play
External	MptkExternalPlayback — file:// / http(s):// playback, optionally connect the InputAdapter (ConnectInputAdapterOnPlay)
Sample	Assets/MIDI/Samples/MPTK/Scripts/MptkWriterExternalSample.cs (generate / Join / temp .mid / URI)

```

#if FEATURE_USE_MPTK && MPTK_PRO
using jp.kshoji.unity.midi.mptk;
using UnityEngine;

public sealed class MyWriterExternalSetup : MonoBehaviour
{
    public MidiSequenceAsset sequence;

    void Start()
    {
        var writer = gameObject.AddComponent<MptkWriterBridge>();
        var external = gameObject.AddComponent<MptkExternalPlayback>();
        writer.LoadFromSequenceAsset(sequence);
        var path = writer.WriteToTempFile();
        external.ConnectInputAdapterOnPlay = true;
        external.PlayFile(path);
    }
}
#endif

```

Playlist / SoundFont / delayed dispatch / channel theater

Component	Demo basis	Pro guard	Sample
MptkListPlayerBridge	TestMidiListPlayer	MidiListPlayer → MPTK_PRO	MptkListPlayerSample.cs
MptkSoundFontLoader	TestLoadSF	runtime Load → MPTK_PRO	MptkPhaseDUilitiesSample.cs
MptkDelayedNoteDispatcher	CatchMusic	none (OnEventNotesMidi)	same
MptkFilePlayerChannels	MidiChannel*	PlayDirect / StopDirect → MPTK_PRO	same

List: SetPlaylist / AddMidi / PlayAtIndex / OverlayTimeMs . You can connect to the InputAdapter at song start via ConnectInputAdapterOnSongStart .

SoundFont: Load a URL / file:// / StreamingAssets / built-in name. Verify the applied name with MptkUtility.ValidateSetup(..., expectedSoundFontName: "...") .

Delayed: Queue a muted FilePlayer's OnEventNotesMidi by ms or tick offset and send it to the Stream or MidiManager . No visual demo is bundled — contract only.

Channels: SetChannelEnabled / SetSoloChannel / SetDrumsOnly / SetSustain / PlayDirect / StopDirect / Panic .

Spatializer / Visual Scripting / Timeline

Component	Demo basis	Pro / Define	Sample
MptkSpatializerHost	SimplestMidiSpatializer	MidiSpatializer → MPTK_PRO	MptkSpatializerSample.cs
MptkSpatializerLayout	—	deviceId / position SO	—
MptkDistanceAudioSettings	—	DistanceAttenuation + Orientation (Pro)	MCP midi-mptk-distance-audio
VS nodes (Pipeline etc.)	P2	FEATURE_USE_VISUALSCRIPTING	Register via Window menu
MptkFilePlayerMarker + Receiver	Seek / InnerLoop	FEATURE_USE_TIMELINE	MptkTimelineMarkerExample.cs

Assign the Maestro Pro **MidiSpatializer** prefab to the host. Audible 3D also needs a Unity spatializer plugin. Per-synth notes inject into `MidiManager` as `deviceIdPrefix:index` (or a Layout suffix).

Layout SO: `Create > MIDI > MPTK > Spatializer Layout` (`MptkSpatializerLayout`).

Distance attenuation vs Spatializer

These are **different** Maestro features:

Feature	Component / API	Role
Spatializer (Track/Channel)	MptkSpatializerHost + Maestro MidiSpatializer	One MIDI file → many synths (per track or channel), each parented to a 3D anchor
Distance attenuation	MptkDistanceAudioSettings / MPTK_DistanceAttenuation	Single synth volume vs listener distance (min/max, pause-on-max)
Orientation (Pro)	same settings / MPTK_Orientation	Pan + front/back filter from angle to MPTK_AudioListener

Use Spatializer for multi-instrument layouts. Use `MptkDistanceAudioSettings` (or the host’s optional **Distance / Orientation on Arrange**) when a Stream/File player (or each spatial synth) should react to listener distance/angle. MCP: `midi-mptk-distance-audio`.

Chord progression generation vs chord recognition

Feature	API	Role
Progression generation (MPTK Pro)	MptkChordProgressionPlayer / midi-mptk-chord-progression	Plays Maestro emotional progression presets on MidiStreamPlayer
Chord recognition (kit)	ChordRecognition / midi-chord-state	Names a chord from currently held input notes

These are not interchangeable: generation drives audio; recognition inspects input.

Drum preset catalog: `midi://mptk/soundfont` and `midi-mptk-bank-program` with `action=list-drums`.

`MptkIntegrationSampleScene` already adds optional Pipeline / Writer / InnerLoop GUI toggles (plus Effects / Voice / Distance / Chord).

Sample scene / output preset / editor preview

Dedicated sample scene

Item	Path
Scene	Assets/MIDI/Samples/MPTK/Scenes/MptkIntegrationSampleScene.unity
Script	Assets/MIDI/Samples/MPTK/Scripts/MptkIntegrationSampleScene.cs

When `FEATURE_USE_MPTK` is enabled, you can try the following from the GUI:

- `MptkBootstrap` setup / validation / Panic
- Note sends via `MidiOutputRoutingPreset`
- SMF preview with `SmfPlayer` + `MptkSmfPlayerOutput`
- Pipeline (arp) / Writer (demo notes or SequenceAsset) / InnerLoop region / Effects / Voice / Distance / Chord

In the Inspector, assign `Assets/MIDI/Scripts/Integrations/MPTK/Presets/MptkVirtualOutput.preset.asset` to `outputPreset`.

Kit-wide `outputDeviceId` preset

With `MidiOutputRoutingPreset` (`Assets > Create > MIDI > Output Routing Preset`) and the `MidiOutputRouting` helper, you can share the MPTK output destination across components.

Available on send components that have `outputDeviceId` / `outputPreset` fields.

Resolution order: `outputDeviceId` (explicit) > `outputPreset` > **first output device**

Bundled preset:

- `Assets/MIDI/Scripts/Integrations/MPTK/Presets/MptkVirtualOutput.preset.asset` — routes to `mptk:internal`

```
using jp.kshoji.unity.midi.foundation;

// Kit send example
MidiOutputRouting.CreateBuilder(outputDeviceId, outputPreset, group)
    .Channel(channel)
    .NoteOn(60, 100);
```

Editor SMF preview (`Window > MIDI > SMF Preview`)

Added a **Preview Audio (MPTK)** button to the SMF Preview window (requires `FEATURE_USE_MPTK`).

1. Load a `.mid`
2. Click **Preview Audio (MPTK)** → enters Play Mode and `SmfPlayer` plays through the MPTK virtual output
3. **Stop Preview** to stop

Internal components:

Component	Role
<code>SmfPreviewPlaybackRequest</code>	Playback request from editor → Play Mode
<code>SmfPreviewPlaybackHost</code>	Launches <code>SmfPlayer</code> in Play Mode
<code>MptkSmfPreviewPlaybackHook</code>	Pre-initializes MPTK Bootstrap

Clock sync / loop region / MIDI 2.0 input / MPE / Visual Scripting

Clock sync — MptkClockSyncBridge

Links an external `MidiClockSync` with MPTK playback (`SmfPlayer` / `MidiFilePlayer`).

Mode	Behavior
Follow	Reflects the estimated BPM into <code>SmfPlayer.tempoBpm</code> / <code>MidiFilePlayer.MPTK_Tempo</code>
Step	Advances <code>SmfPlayer</code> one beat per external Clock beat
Free	Ignores the external Clock

Enabling `syncTransportToClock` starts/stops SMF / MPTK file playback in sync with external Start / Stop.

```
#if FEATURE_USE_MPTK
using jp.kshoji.unity.midi;
using jp.kshoji.unity.midi.mptk;
using UnityEngine;

public sealed class MyClockBridge : MonoBehaviour
{
    public MidiClockSync clockSync;
    public SmfPlayer smfPlayer;

    void Awake()
    {
        gameObject.AddComponent<MptkBootstrap>().EnsureReady();
        gameObject.AddComponent<MptkSmfPlayerOutput>().Configure();
        gameObject.AddComponent<MptkClockSyncBridge>();
    }
}
#endif
```

To inject Clock events from MPTK callbacks into `MidiManager`, the existing `MptkMidiManagerInputAdapter` forwards Timing Clock / Start / Stop / Continue.

Loop region — MptkInnerLoopController (Maestro Pro)

`MptkClockSyncBridge` handles **external Clock** tempo/transport sync. In contrast, `MptkInnerLoopController` wraps the **in-player loop during MPTK FilePlayer / ExternalPlayer playback** (`MPTK_InnerLoop`). It is a different engine from `SmfPlayer` or DSP Scheduler loops.

Item	Detail
Defines	<code>FEATURE_USE_MPTK</code> + <code>MPTK_PRO</code> (<code>MPTK_InnerLoop</code>)
Component	<code>MptkInnerLoopController</code> — Start / Resume / End / Max / Finished
Events	<code>OnLoopStart</code> / <code>OnLoopResume</code> / <code>OnLoopExit</code> (main thread). <code>PhaseFilter</code> on the MIDI thread (no Unity APIs)
Helpers	<code>MeasureToTick</code> / <code>SetLoopByMeasure</code> (time signature → tick)
Sample	<code>Assets/MIDI/Samples/MPTK/Scripts/MptkInnerLoopSample.cs</code>

Because `MPTK_InnerLoop` is cleared on MIDI load, by default the parameters are re-applied on `OnEventStartPlayMidi` (`ReapplyOnStartPlay`).

Free fallback: Without Pro, a coarse region restart is possible with `MPTK_MidiLoaded.MPTK_TickStart` / `MPTK_TickEnd` + `MPTK_MidiAutoRestart` (equivalent to the `MidiLoop` demo). Use `InnerLoop` when you need accuracy and phase callbacks.

```
#if FEATURE_USE_MPTK && MPTK_PRO
using jp.kshoji.unity.midi.mptk;
using MidiPlayerTK;
using UnityEngine;

public sealed class MyInnerLoopSetup : MonoBehaviour
{
    public MidiFilePlayer filePlayer;

    void Start()
    {
        var loop = gameObject.AddComponent<MptkInnerLoopController>();
        loop.Source = filePlayer;
        // Start → Resume ... → End (Max times); Max=0 is infinite
        loop.SetLoop(start: 0, resume: 480 * 4, end: 480 * 16, max: 3);
        loop.OnLoopExit.AddListener(() => Debug.Log("chorus loop done"));
        filePlayer.MPTK_Play();
    }
}
#endif
```

MIDI 2.0 input adapter — `MptkMidi2ManagerInputAdapter`

Injects MPTK player callbacks into `Midi2Manager` as **MIDI 2.0 virtual input** (default deviceId: `mptk2:internal`).

```

#if FEATURE_USE_MPTK
using jp.kshoji.unity.midi.mptk;

MptkUtility.CreateMidi2ManagerInputAdapter(
    filePlayer,
    streamPlayer,
    deviceId: MptkUtility.DefaultMidi2InputDeviceId);
#endif

```

MIDI 1.0 7-bit values are scaled to 16-bit / 32-bit for UMP before injection.

MPE output — MptkMpeOutput

Sends MPE notes to the MPTK virtual device via `MpeManager`. Internally it calls `SetupMpeZone`; member channel assignment is handled by `MpeManager`.

```

#if FEATURE_USE_MPTK
var mpe = gameObject.AddComponent<MptkMpeOutput>();
mpe.ConfigureZone();
mpe.SendNoteOn(60, 100);
#endif

```

Visual Scripting nodes (`FEATURE_USE_MPTK` + `FEATURE_USE_VISUALSCRIPTING`)

Added MPTK-specific nodes to the assembly `jp.kshoji.midi.mptk.visualscripting`.

Node	Category	Description
MPTK Ensure Ready	MIDI/MPTK	<code>MptkBootstrap.EnsureReady()</code>
MPTK Panic	MIDI/MPTK	All Notes Off
MPTK Send Routed Note On	MIDI/MPTK	Note On via <code>MidiOutputRoutingPreset</code>
MPTK Configure MPE Zone	MIDI/MPTK	<code>MptkMpeOutput.ConfigureZone()</code>
MPTK Pipeline Configure	MIDI/MPTK	Pipeline arp / drop patch / tempo
MPTK Writer Play	MIDI/MPTK	<code>MptkWriterBridge.Play()</code>
MPTK External Play	MIDI/MPTK	<code>MptkExternalPlayback.Play(uri)</code>
MPTK InnerLoop Apply	MIDI/MPTK	<code>MptkInnerLoopController.SetLoop</code>
MPTK List Play	MIDI/MPTK	<code>MptkListPlayerBridge.Play</code> / <code>PlayAtIndex</code>
MPTK SoundFont Load	MIDI/MPTK	<code>MptkSoundFontLoader.Load</code>
MPTK Spatializer Play	MIDI/MPTK	<code>MptkSpatializerHost.Play</code>
MPTK Effects Apply	MIDI/MPTK	SoundFont filter / reverb / chorus
MPTK Voice Lifecycle	MIDI/MPTK	Pause / resume voices
MPTK Distance Audio Apply	MIDI/MPTK	Distance / orientation knobs
MPTK Chord Progression	MIDI/MPTK	Play / stop progression presets
MPTK Global Settings	MIDI/MPTK	<code>SetRunInBackground</code>

First-time setup:

1. Enable `FEATURE_USE_MPTK` and `FEATURE_USE_VISUALSCRIPTING`
2. Run **Window > MIDI > Visual Scripting > Register MPTK Nodes**
3. Use **MIDI/MPTK** category nodes in a Script Graph

Documentation examples

Example	Path
Clock sync	<code>Assets/MIDI/Samples/DocumentationExamples/MptkClockSyncExample.cs</code>
MIDI 2.0 input	<code>Assets/MIDI/Samples/DocumentationExamples/MptkMidi2InputAdapterExample.cs</code>
MPE output	<code>Assets/MIDI/Samples/DocumentationExamples/MptkMpeOutputExample.cs</code>
Event Pipeline	<code>Assets/MIDI/Samples/DocumentationExamples/MptkMidiEventPipelineExample.cs</code>
Writer / External	<code>Assets/MIDI/Samples/DocumentationExamples/MptkWriterExternalExample.cs</code>
InnerLoop	<code>Assets/MIDI/Samples/DocumentationExamples/MptkInnerLoopExample.cs</code>
List / SF / Delayed / Channels	<code>MptkListPlayerExample.cs</code> / <code>MptkSoundFontLoaderExample.cs</code> / <code>MptkDelayedNoteDispatcherExample.cs</code> / <code>MptkFilePlayerChannelsExample.cs</code>
Spatializer	<code>Assets/MIDI/Samples/DocumentationExamples/MptkSpatializerExample.cs</code>
Timeline Marker	<code>Assets/MIDI/Samples/DocumentationExamples/MptkTimelineMarkerExample.cs</code>

The following samples are provided under `Assets/MIDI/Samples/DocumentationExamples/` :

- **MIDI 1.0 → MPTK (virtual output sink)**

`Assets/MIDI/Samples/DocumentationExamples/MptkVirtualOutputSinkExample.cs`

- **MPTK → MIDI 1.0 (inject into MidiManager as virtual input)**

`Assets/MIDI/Samples/DocumentationExamples/MptkToMidiManagerInputExample.cs`

- **Bootstrap + SmfPlayer + validation**

`Assets/MIDI/Samples/DocumentationExamples/MptkBootstrapExample.cs`

- **Clock sync**

`Assets/MIDI/Samples/DocumentationExamples/MptkClockSyncExample.cs`

- **Pre-synthesis event pipeline**

`Assets/MIDI/Samples/DocumentationExamples/MptkMidiEventPipelineExample.cs`

- **Writer / External round-trip**

`Assets/MIDI/Samples/DocumentationExamples/MptkWriterExternalExample.cs`

- **InnerLoop (loop region)**

`Assets/MIDI/Samples/DocumentationExamples/MptkInnerLoopExample.cs`

- **List / SoundFont / Delayed / Channels**

`Assets/MIDI/Samples/DocumentationExamples/MptkListPlayerExample.cs` and related

- **Spatializer**

`Assets/MIDI/Samples/DocumentationExamples/MptkSpatializerExample.cs`

- **Timeline Marker Seek / InnerLoop**

`Assets/MIDI/Samples/DocumentationExamples/MptkTimelineMarkerExample.cs`

- **MIDI 2.0 input adapter**

`Assets/MIDI/Samples/DocumentationExamples/MptkMidi2InputAdapterExample.cs`

- **MPE output**

`Assets/MIDI/Samples/DocumentationExamples/MptkMpeOutputExample.cs`

Sample scenes (integrated)

The built-in sample scenes also include optional MPTK integration:

- MIDI 1.0 sample scene script:

`Assets/MIDI/Samples/Scripts/MidiSampleScene.cs`

(adds a toggle to use an MPTK-backed virtual output device)

- MIDI 2.0 sample scene script:

`Assets/MIDI/Samples/Scripts/Midi2SampleScene.cs`

(adds a toggle to *mirror* MIDI 2.0 sends into an MPTK MIDI 1.0 virtual sink)

Choosing IDs, groups, and routing

Recommended convention:

- Use a clearly virtual prefix (e.g., `mptk:internal`, `virtual:sequencer`, `test:device`).
- Use `group = 0` unless you intentionally model multiple groups.

Because the virtual device appears in the device sets, you can build UI that lets users select it just like a hardware device.

Troubleshooting

“It compiles in Editor but fails on CI / another machine”

- Ensure `FEATURE_USE_MPTK` is only enabled when the MPTK asset is actually present.

“No events are received”

- Confirm the virtual device was registered as **input** (for injected events to look like input devices).
- Confirm your handler is registered with `MidiManager`.
- Confirm `deviceId` and `group` match what you expect.

“No sound from MPTK sink”

- Confirm the virtual device was registered as **output**.
- Confirm the sink’s `deviceId` matches the device you are sending to.
- Confirm MPTK global setup/resources are valid in your project (SoundFont / configuration).
- Use `MptkUtility.ValidateSetup()` or `MptkBootstrap.Validate()` to check the SoundFont / AudioSource / virtual device registration.

“No sound from SmfPlayer to MPTK”

- Add `MptkSmfPlayerOutput` to the same GameObject (or a child) and call `Configure()`.
- Confirm `SmfPlayer.outputDeviceId` is `mptk:internal` (or the ID set by Bootstrap).
- `MptkBootstrap.EnsureReady()` must complete before entering Play mode.

“No sound from Scriptable Audio UMP to MPTK”

- Confirm that both `FEATURE_SCRIPTABLE_AUDIO` and `FEATURE_USE_MPTK` are enabled.
- Add `MptkDspUmpSequenceOutput` and confirm `MidiDspUmpMidi2OutBridge` is enabled (Bootstrap UMP Out or manual wiring).
- Confirm `MidiManager.InitializeMidi2()` is called.

“Sound lingers even after Panic”

- Call `MptkBootstrap.Panic()` or `MptkMidiDevice.PanicAll()` directly.
- When sending to both hardware and MPTK, Panic is required for each destination.

Related docs

- [MIDI 1.0 \(MidiManager\)](#)
- [Unity ecosystem integration — UMP sequence](#)
- [Samples](#)